

ECOM EXPERTS

Ecom filtersearch

Manual de usuario, técnico y de desarrollo

ecom_filtersearch · 08/09/2026

Contenido

Descripción

Características

Cómo Funciona

Flujo Automático

Ventajas vs Sistema Anterior

Archivos Creados

Classes (Nuevas)

Controllers

Cron

Configuración

1. Requisitos

2. Configuración Automática

3. Configurar Cron Job (Recomendado)

Uso

Uso Automático (Recomendado)

Gestión Manual desde Back Office

Base de Datos

Tablas Creadas Automáticamente

Logs

Ubicación

Ejemplo de Log

Costos

Comparación de Costos (Ejemplo con 20MB de contexto)

Ahorro con Cache

Troubleshooting

El cache no se crea

Cache expira muy rápido

Archivo no se sube

Cron no funciona

 Seguridad

 Monitoreo

Verificar Estado del Sistema

Métricas Importantes

 Soporte

 Changelog

v1.0.0 (2026-01-13)

 Resumen de Cambios

 Soporte Multi-Tienda

Problema Resuelto

Identificador de Tienda

Dónde se Aplica

 Logging Completo

Nuevo Sistema de Logs

Archivos de Log

Qué se Registra

 Archivos Modificados

1. classes/GeminiFileManager.php

2. classes/GeminiCacheManager.php

3. classes/GeminiLargeContextManager.php

 Estructura de Base de Datos

Tabla ps_ecom_gemini_files

Tabla ps_ecom_gemini_cache

 Ejemplo de Uso Multi-Tienda

Escenario: 3 Tiendas con la Misma API Key

En Gemini Console

 Verificación de Logs

Ver Logs en Tiempo Real

Buscar Errores

Ver Solo Operaciones de Upload

Ver Solo Operaciones de Cache

 Checklist de Verificación

Multi-Tienda

Logging

 Beneficios

Multi-Tienda

Logging

 Próximos Pasos

 Ejemplo de Log Completo

 Resumen de la Solución

 Archivos Creados

1. Classes (3 archivos nuevos)

2. Controllers (1 archivo nuevo)

3. Cron (1 archivo nuevo)

4. Documentación (2 archivos)

 Modificaciones en Archivos Existentes

controllers/front/assist.php

 Base de Datos

Tablas Creadas Automáticamente

 Cómo Funciona (Flujo Automático)

Primera Vez (Archivo > 3MB)

Consultas Siguientes

Renovación Automática

 Ahorro de Costos

Ejemplo con archivo de 20MB

 Configuración del Cron Job

Linux / macOS

Windows

 Uso Manual (Opcional)

Ver Estadísticas

Actualizar Cache Manualmente

Limpiar Cache



Logs

Ubicaciones

Ejemplo de Log Exitoso



Ventajas de la Solución



Responde a tus Requisitos



Beneficios Adicionales



Verificación

Comprobar que Funciona



Soporte



Documentación Completa



¡Listo para Usar!



Resumen



Nuevas Características



Soporte Multi-Tienda



Logging Completo



Cambios en Base de Datos

Tabla ps_ecom_gemini_files

Tabla ps_ecom_gemini_cache



Proceso de Actualización

Automático (Recomendado)

Manual (Si es necesario)



Verificación Post-Actualización

1. Verificar Versión

2. Verificar Base de Datos

3. Verificar Logs

4. Probar Funcionalidad



Logs del Upgrade



Notas Importantes

Compatibilidad

Tablas Opcionales

Múltiples Tiendas

Troubleshooting

Error: "Column already exists"

Error: "Table doesn't exist"

Registros no actualizados

Impacto en Rendimiento

Documentación Adicional

Checklist Post-Upgrade

 Descripción

Sistema avanzado para manejar archivos de catálogo grandes (20MB+) usando **Gemini File API** y **Context Caching**.

 Características

- ✔ Soporta archivos hasta **2GB** (vs 3MB límite anterior)
- ✔ **Reduce costos en 75%** para tokens cacheados
- ✔ Cache válido por **1 hora** (renovable automáticamente)
- ✔ Archivos permanecen **48 horas** en Gemini
- ✔ Actualización automática **diaria** vía cron
- ✔ Gestión manual desde back office

 Cómo Funciona

Flujo Automático

- Detección:** Si `llms.txt` > 3MB, se activa automáticamente
- Upload:** El archivo se sube a Gemini File API (primera vez)
- Cache:** Se crea un cache del contexto (válido 1 hora)
- Uso:** Todas las consultas usan el cache (75% más barato)
- Renovación:** Cache se renueva automáticamente cada 30 min si hay actividad
- Actualización:** Cron job actualiza el cache cada 24h

Ventajas vs Sistema Anterior

Característica	Anterior	Nuevo
Tamaño máximo	3MB	2GB
Costo por consulta	100%	25% (con cache)
Velocidad	Normal	Más rápida
Contexto completo	✗ (RAG)	✔ (Todo el catálogo)

Archivos Creados

Classes (Nuevas)

1. **GeminiFileManager.php** - Gestiona uploads de archivos grandes a Gemini - Métodos: `uploadFile()`, `getFile()`, `deleteFile()`, `listFiles()`
2. **GeminiCacheManager.php** - Gestiona caches de contexto - Métodos: `createCache()`, `getCache()`, `updateCache()`, `deleteCache()`
3. **GeminiLargeContextManager.php** (Orquestador principal) - Coordina todo el proceso - Método principal: `getOrCreateContext($filePath)`

Controllers

4. **controllers/front/cachemanagement.php** - API para gestión manual desde back office - Acciones: `get_stats`, `update_cache`, `clear_cache`, `test_upload`

Cron

5. **cron/update_cache.php** - Script para actualización automática diaria - Ejecutar: `php cron/update_cache.php`

Configuración

1. Requisitos

- PrestaShop 1.7+
- Módulo `ecom_filtersearch` instalado
- API Key de Gemini configurada
- Modelo: `gemini-1.5-pro-002` o `gemini-1.5-flash-002`

2. Configuración Automática

No se requiere configuración adicional. El sistema se activa automáticamente cuando: - `llms.txt` existe - Tamaño > 3MB - Provider = Gemini

3. Configurar Cron Job (Recomendado)

Añade a tu crontab para actualización diaria a las 2 AM:

```
0 2 * * * php /ruta/a/prestashop/modules/ecom_filtersearch/cron/update_cache.php
```

Windows (Programador de tareas):


```
Programa: php.exe
Argumentos: "Z:\ruta\a\prestashop\modules\ecom_filtersearch\cron\update_cache.php"
Frecuencia: Diariamente a las 2:00 AM
```

Uso

Uso Automático (Recomendado)

El sistema funciona automáticamente. No necesitas hacer nada especial.

1. Usuario abre el asistente en la home page
2. Sistema detecta `llms.txt` > 3MB
3. Crea/usa cache automáticamente
4. Usuario recibe respuestas con contexto completo

Gestión Manual desde Back Office

Ver Estadísticas

```
// Llamar desde consola del navegador en back office
fetch('/module/ecom_filtersearch/cachemanagement?action=get_stats')
  .then(r => r.json())
  .then(data => console.log(data));
```

Respuesta:

```
{
  "success": true,
  "stats": {
    "has_active_cache": true,
    "cache_name": "cachedContents/abc123xyz",
    "cache_created": "2026-01-13 09:00:00",
    "cache_expires": "2026-01-13 10:00:00",
    "total_files": 1,
    "total_size_mb": 25.5,
    "llms_file_exists": true,
    "llms_file_size_mb": 25.5,
    "llms_file_modified": "2026-01-13 08:00:00"
  }
}
```

Actualizar Cache Manualmente

```
fetch('/module/ecom_filtersearch/cachemanagement?action=update_cache')
  .then(r => r.json())
  .then(data => console.log(data));
```

Limpiar Cache

```
fetch('/module/ecom_filtersearch/cachemanagement?action=clear_cache')
  .then(r => r.json())
  .then(data => console.log(data));
```

Test de Upload

```
fetch('/module/ecom_filtersearch/cachemanagement?action=test_upload')
  .then(r => r.json())
  .then(data => console.log(data));
```



Base de Datos

Tablas Creadas Automáticamente

ps_ecom_gemini_cache

```
CREATE TABLE `ps_ecom_gemini_cache` (
  `id` INT(11) AUTO_INCREMENT,
  `cache_name` VARCHAR(255) NOT NULL,
  `display_name` VARCHAR(255) NOT NULL,
  `model` VARCHAR(100) NOT NULL,
  `expire_time` DATETIME NULL,
  `created_at` DATETIME NOT NULL,
  `updated_at` DATETIME NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `cache_name` (`cache_name`)
);
```

ps_ecom_gemini_files

```
CREATE TABLE `ps_ecom_gemini_files` (
  `id` INT(11) AUTO_INCREMENT,
  `file_name` VARCHAR(255) NOT NULL,
  `uri` VARCHAR(500) NOT NULL,
  `mime_type` VARCHAR(100) NOT NULL,
  `size_bytes` BIGINT NOT NULL,
  `local_path` VARCHAR(500) NOT NULL,
  `created_at` DATETIME NOT NULL,
  `expires_at` DATETIME NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `file_name` (`file_name`)
);
```

 Logs

Ubicación

- **Asistente:** logs/assist_debug_YYYY-MM-DD.log
- **Cron:** logs/cache_cron_YYYY-MM-DD.log
- **Gestión:** logs/cache_management_YYYY-MM-DD.log

Ejemplo de Log

```
[09:00:00] === Large Context Manager: Getting or creating context ===
[09:00:00] File size: 25,500,000 bytes (25.5 MB)
[09:00:01] Creating new cache...
[09:00:01] Step 1: Uploading file to Gemini File API...
[09:00:15] File uploaded successfully: files/abc123xyz
[09:00:15] File URI: https://generativelanguage.googleapis.com/v1beta/files/abc123xyz
[09:00:15] Step 2: Creating cached context...
[09:00:20] Cache created successfully: cachedContents/xyz789abc
[09:00:20] SUCCESS: Cache updated
```

 Costos

Comparación de Costos (Ejemplo con 20MB de contexto)

Archivo de 20MB ≈ 5M tokens

Método	Tokens Input	Costo por Consulta*	Costo 1000 Consultas*
Anterior (RAG)	~100K tokens	\$0.0125	\$12.50
Nuevo (Sin Cache)	5M tokens	\$0.625	\$625.00
Nuevo (Con Cache)	5M tokens (cached)	\$0.156	\$156.00

*Precios aproximados con Gemini 1.5 Pro

Ahorro con Cache

- **75% de ahorro** en tokens cacheados
- Cache válido 1 hora
- Renovación automática si hay actividad

Troubleshooting

El cache no se crea

Problema: Error al crear cache

Soluciones: 1. Verificar API Key válida 2. Verificar modelo soporta caching (`gemin-1.5-pro-002` o `gemin-1.5-flash-002`) 3. Revisar logs en `logs/assist_debug_*.log` 4. Probar upload manual: `action=test_upload`

Cache expira muy rápido

Problema: Cache se invalida antes de 1 hora

Solución: El cache se renueva automáticamente cada 30 min si hay actividad. Si no hay consultas, expira en 1 hora (comportamiento normal).

Archivo no se sube

Problema: Error al subir archivo a Gemini

Soluciones: 1. Verificar tamaño < 2GB 2. Verificar permisos de lectura en `llms.txt` 3. Verificar conexión a internet 4. Revisar logs para errores específicos

Cron no funciona

Problema: Cron job no actualiza el cache

Soluciones: 1. Verificar crontab configurado correctamente 2. Ejecutar manualmente: `php cron/update_cache.php` 3. Revisar logs en `logs/cache_cron_*.log` 4. Verificar permisos de ejecución

Seguridad

- ☒ Archivos en Gemini expiran en 48 horas
 - ☒ Cache expira en 1 hora (renovable)
 - ☒ Limpieza automática de archivos antiguos
 - ☒ API endpoints protegidos (solo admin)
 - ☒ Logs detallados para auditoría
-

Monitoreo

Verificar Estado del Sistema

```
require_once 'classes/GeminiLargeContextManager.php';

$apiKey = Configuration::get('ECOM_FS_IA_API_KEY');
$model = 'gemini-1.5-pro-002';

$manager = new GeminiLargeContextManager($apiKey, $model);
$stats = $manager->getStats();

print_r($stats);
```

Métricas Importantes

- `has_active_cache` : ¿Hay cache activo?
 - `cache_created` : Fecha de creación
 - `cache_expires` : Fecha de expiración
 - `total_files` : Archivos en Gemini
 - `total_size_mb` : Tamaño total
-

Soporte

Para problemas o preguntas:

1. Revisar logs en `modules/ecom_filtersearch/logs/`
 2. Ejecutar test: `action=test_upload`
 3. Verificar configuración de API Key
 4. Contactar soporte técnico
-

Changelog

v1.0.0 (2026-01-13)

- 🌟 Implementación inicial
- ✅ Soporte para archivos hasta 2GB
- ✅ Context Caching automático
- ✅ Cron job para actualizaciones diarias
- ✅ Gestión manual desde back office
- ✅ Ahorro de 75% en costos con cache

Multi-Tienda + Logging Completo - Implementación Completada

Resumen de Cambios

Se han implementado dos mejoras críticas al sistema de Gemini Large Context Manager:

1.  **Soporte Multi-Tienda:** Diferenciación de archivos y caches entre múltiples instalaciones de PrestaShop
2.  **Logging Completo:** Registro detallado de todas las operaciones de cache y upload

Soporte Multi-Tienda

Problema Resuelto

Antes: Si usabas la misma API Key en múltiples tiendas PrestaShop, los archivos y caches se mezclaban y no había forma de diferenciarlos.

Ahora: Cada tienda tiene su propio identificador único basado en: - **Dominio de la tienda** (ej: `mitienda.com`) - **Nombre de la tienda** (ej: `Mi Tienda Online`)

Identificador de Tienda

Formato: `{dominio}_{nombre_tienda}`

Ejemplo: - Tienda 1: `mitienda-com_mi-tienda-online` - Tienda 2: `otratienda-es_otra-tienda`

Dónde se Aplica

1. Nombres de Archivos en Gemini

Antes:

```
Product Catalog - 2026-01-13 10:00
```

Ahora:

```
[mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
```

2. Nombres de Caches en Gemini

Antes:

```
Product Catalog - 2026-01-13 10:00
```

Ahora:

```
[mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
```

3. Base de Datos

Tabla `ps_ecom_gemini_files` - Nuevas columnas: - `shop_identifier` - Identificador único (ej: `mitienda-com_mi-tienda-online`) - `shop_domain` - Dominio (ej: `mitienda.com`) - `shop_name` - Nombre (ej: `Mi Tienda Online`)

Tabla `ps_ecom_gemini_cache` - Nuevas columnas: - `shop_identifier` - Identificador único - `shop_domain` - Dominio - `shop_name` - Nombre



Logging Completo

Nuevo Sistema de Logs

Todos los logs se guardan en: `modules/ecom_filtersearch/logs/`

Archivos de Log

1. `assist_debug_YYYY-MM-DD.log` - Logs del asistente y operaciones generales
2. `cache_cron_YYYY-MM-DD.log` - Logs del cron job de actualización
3. `cache_management_YYYY-MM-DD.log` - Logs de gestión manual

Qué se Registra



Upload de Archivos

```
[10:00:00] === GeminiFileManager initialized ===
[10:00:00] Shop Identifier: mitienda-com_mi-tienda-online
[10:00:01] === UPLOAD FILE START ===
[10:00:01] File path: /path/to/llms.txt
[10:00:01] File size: 25,500,000 bytes (25.5 MB)
[10:00:01] MIME type: text/plain
[10:00:01] Display name: [mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
[10:00:01] Multipart payload size: 25,501,234 bytes
[10:00:01] Upload URL: https://generativelanguage.googleapis.com/v1beta/files?key=***API_KEY***
[10:00:01] Executing CURL upload to Gemini File API...
[10:00:15] Upload completed in 14.23 seconds
[10:00:15] HTTP Response Code: 200
[10:00:15] SUCCESS: File uploaded to Gemini
[10:00:15] File name: files/abc123xyz
[10:00:15] File URI: https://generativelanguage.googleapis.com/v1beta/files/abc123xyz
[10:00:15] File size: 25,500,000 bytes
[10:00:15] === UPLOAD FILE SUCCESS ===
```

Creación de Cache

```
[10:00:16] === GeminiCacheManager initialized ===
[10:00:16] Shop Identifier: mitienda-com_mi-tienda-online
[10:00:16] === CREATE CACHE START ===
[10:00:16] File URI: https://generativelanguage.googleapis.com/v1beta/files/abc123xyz
[10:00:16] Model: gemini-1.5-pro-002
[10:00:16] TTL: 3600 seconds
[10:00:16] Display name: [mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
[10:00:16] Executing CURL request to create cache...
[10:00:20] Cache creation completed in 4.12 seconds
[10:00:20] HTTP Response Code: 200
[10:00:20] SUCCESS: Cache created in Gemini
[10:00:20] Cache name: cachedContents/xyz789abc
[10:00:20] Display name: [mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
[10:00:20] Expires at: 2026-01-13T11:00:20Z
[10:00:20] Storing cache info in database...
[10:00:20] Storing cache with shop identifier: mitienda-com_mi-tienda-online
[10:00:20] === CREATE CACHE SUCCESS ===
```

Proceso Completo

```
[10:00:00] === GeminiLargeContextManager initialized ===
[10:00:00] API Key: Present (39 chars)
[10:00:00] Model: gemini-1.5-pro-002
[10:00:00] === GeminiFileManager initialized ===
[10:00:00] Shop Identifier: mitienda-com_mi-tienda-online
[10:00:00] === GeminiCacheManager initialized ===
[10:00:00] Shop Identifier: mitienda-com_mi-tienda-online
[10:00:00] === Large Context Manager: Getting or creating context ===
[10:00:00] File size: 25,500,000 bytes (25.5 MB)
[10:00:00] Creating new cache...
[10:00:00] Step 1: Uploading file to Gemini File API...
[... upload logs ...]
[10:00:15] File uploaded successfully: files/abc123xyz
[10:00:15] File URI: https://generativelanguage.googleapis.com/v1beta/files/abc123xyz
[10:00:15] Storing file info in database...
[10:00:15] Storing file with shop identifier: mitienda-com_mi-tienda-online
[10:00:15] File info stored successfully
[10:00:15] Step 2: Creating cached context...
[... cache creation logs ...]
[10:00:20] Cache created successfully: cachedContents/xyz789abc
[10:00:20] Cleaning up old resources...
[10:00:20] Cleanup completed
```


Archivos Modificados

1. `classes/GeminiFileManager.php`

Cambios: -  Añadido `private $shopIdentifier` -  Método `getShopIdentifier()` para obtener identificador único -  Identificador se añade al `display_name` de archivos: `[shop_id] filename` -  Logging detallado de todo el proceso de upload -  Registro de tiempos de ejecución -  Logs de errores con respuestas truncadas

Logs añadidos: - Inicialización con shop identifier - Tamaño de archivo en bytes y MB - MIME type detectado - Display name con shop identifier - Tamaño de payload multipart - URL de upload (con API key enmascarada) - Tiempo de upload - Información completa del archivo subido

2. `classes/GeminiCacheManager.php`

Cambios: -  Añadido `private $shopIdentifier` -  Método `getShopIdentifier()` para obtener identificador único -  Identificador se añade al `display_name` de caches: `[shop_id] cache_name` -  Logging detallado de creación de cache -  Nuevas columnas en tabla `ps_ecom_gemini_cache` : - `shop_identifier` - `shop_domain` - `shop_name`

Logs añadidos: - Inicialización con shop identifier - File URI y modelo - TTL configurado - Display name con shop identifier - Tiempo de creación de cache - Información completa del cache creado - Confirmación de almacenamiento en BD

3. `classes/GeminiLargeContextManager.php`

Cambios: -  Logging de inicialización con API key y modelo -  Nuevas columnas en tabla `ps_ecom_gemini_files` : - `shop_identifier` - `shop_domain` - `shop_name` -  Índice en `shop_identifier` para búsquedas rápidas -  Logs de almacenamiento en base de datos

Logs añadidos: - Inicialización con longitud de API key - Modelo configurado - Confirmación de almacenamiento de file info - Shop identifier usado

Estructura de Base de Datos

Tabla `ps_ecom_gemini_files`

```
CREATE TABLE `ps_ecom_gemini_files` (  
  `id` INT(11) NOT NULL AUTO_INCREMENT,  
  `file_name` VARCHAR(255) NOT NULL,  
  `uri` VARCHAR(500) NOT NULL,  
  `mime_type` VARCHAR(100) NOT NULL,  
  `size_bytes` BIGINT NOT NULL,  
  `local_path` VARCHAR(500) NOT NULL,  
  `shop_identifiser` VARCHAR(255) NOT NULL DEFAULT '', -- NUEVO  
  `shop_domain` VARCHAR(255) NOT NULL DEFAULT '', -- NUEVO  
  `shop_name` VARCHAR(255) NOT NULL DEFAULT '', -- NUEVO  
  `created_at` DATETIME NOT NULL,  
  `expires_at` DATETIME NOT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `file_name` (`file_name`),  
  KEY `shop_identifiser` (`shop_identifiser`) -- NUEVO  
);
```

Tabla `ps_ecom_gemini_cache`

```
CREATE TABLE `ps_ecom_gemini_cache` (  
  `id` INT(11) NOT NULL AUTO_INCREMENT,  
  `cache_name` VARCHAR(255) NOT NULL,  
  `display_name` VARCHAR(255) NOT NULL,  
  `model` VARCHAR(100) NOT NULL,  
  `expire_time` DATETIME NULL,  
  `shop_identifiser` VARCHAR(255) NOT NULL DEFAULT '', -- NUEVO  
  `shop_domain` VARCHAR(255) NOT NULL DEFAULT '', -- NUEVO  
  `shop_name` VARCHAR(255) NOT NULL DEFAULT '', -- NUEVO  
  `created_at` DATETIME NOT NULL,  
  `updated_at` DATETIME NOT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `cache_name` (`cache_name`),  
  KEY `shop_identifiser` (`shop_identifiser`) -- NUEVO  
);
```

Ejemplo de Uso Multi-Tienda

Escenario: 3 Tiendas con la Misma API Key

Tienda 1: mitienda.com - "Mi Tienda"

```
Shop Identifier: mitienda-com_mi-tienda
File Display Name: [mitienda-com_mi-tienda] Product Catalog - 2026-01-13 10:00
Cache Display Name: [mitienda-com_mi-tienda] Product Catalog - 2026-01-13 10:00
```

Tienda 2: otratienda.es - "Otra Tienda"

```
Shop Identifier: otratienda-es_otra-tienda
File Display Name: [otratienda-es_otra-tienda] Product Catalog - 2026-01-13 10:00
Cache Display Name: [otratienda-es_otra-tienda] Product Catalog - 2026-01-13 10:00
```

Tienda 3: terceratienda.com - "Tercera Tienda"

```
Shop Identifier: terceratienda-com_tercera-tienda
File Display Name: [terceratienda-com_tercera-tienda] Product Catalog - 2026-01-13 10:00
Cache Display Name: [terceratienda-com_tercera-tienda] Product Catalog - 2026-01-13 10:00
```

En Gemini Console

Verás archivos claramente diferenciados:

```
Files:
- [mitienda-com_mi-tienda] Product Catalog - 2026-01-13 10:00
- [otratienda-es_otra-tienda] Product Catalog - 2026-01-13 10:00
- [terceratienda-com_tercera-tienda] Product Catalog - 2026-01-13 10:00

Cached Contents:
- [mitienda-com_mi-tienda] Product Catalog - 2026-01-13 10:00
- [otratienda-es_otra-tienda] Product Catalog - 2026-01-13 10:00
- [terceratienda-com_tercera-tienda] Product Catalog - 2026-01-13 10:00
```

Verificación de Logs

Ver Logs en Tiempo Real

Linux/Mac:

```
tail -f modules/ecom_filtersearch/logs/assist_debug_*.log
```

Windows (PowerShell):

```
Get-Content "modules\ecom_filtersearch\logs\assist_debug_$(Get-Date -Format 'yyyy-MM-dd').log" -
Wait
```

Buscar Errores

```
grep "ERROR" modules/ecom_filtersearch/logs/*.log
```

Ver Solo Operaciones de Upload

```
grep "UPLOAD FILE" modules/ecom_filtersearch/logs/*.log
```

Ver Solo Operaciones de Cache

```
grep "CREATE CACHE" modules/ecom_filtersearch/logs/*.log
```

Checklist de Verificación

Multi-Tienda

- [x] Shop identifier se genera correctamente
- [x] Shop identifier se añade a nombres de archivos
- [x] Shop identifier se añade a nombres de caches
- [x] Shop identifier se guarda en BD (files)
- [x] Shop identifier se guarda en BD (cache)
- [x] Índices creados para búsquedas rápidas

Logging

- [x] Logs de inicialización de managers
 - [x] Logs de upload completo
 - [x] Logs de creación de cache
 - [x] Logs de tiempos de ejecución
 - [x] Logs de errores con detalles
 - [x] Logs de almacenamiento en BD
 - [x] API keys enmascaradas en logs
-

Beneficios

Multi-Tienda

1. **Claridad:** Sabes exactamente qué archivo/cache pertenece a qué tienda
2. **Gestión:** Puedes identificar y eliminar recursos de tiendas específicas
3. **Debugging:** Logs muestran claramente de qué tienda proviene cada operación
4. **Escalabilidad:** Soporta ilimitadas tiendas con la misma API key

Logging

1. **Transparencia:** Ves exactamente qué está pasando en cada momento
2. **Debugging:** Identifica problemas rápidamente con logs detallados
3. **Auditoría:** Registro completo de todas las operaciones
4. **Monitoreo:** Puedes seguir el progreso de uploads y caches en tiempo real
5. **Troubleshooting:** Tiempos de ejecución ayudan a identificar cuellos de botella

Próximos Pasos

1. **Probar con múltiples tiendas** para verificar diferenciación
2. **Revisar logs** después de primera ejecución
3. **Configurar cron job** para actualizaciones automáticas
4. **Monitorear tamaño de logs** y configurar rotación si es necesario

Ejemplo de Log Completo

Ver archivo: `modules/ecom_filtersearch/logs/assist_debug_2026-01-13.log`

```
[10:00:00] === GeminiLargeContextManager initialized ===
[10:00:00] API Key: Present (39 chars)
[10:00:00] Model: gemini-1.5-pro-002
[10:00:00] === GeminiFileManager initialized ===
[10:00:00] Shop Identifier: mitienda-com_mi-tienda-online
[10:00:00] === GeminiCacheManager initialized ===
[10:00:00] Shop Identifier: mitienda-com_mi-tienda-online
[10:00:00] === Large Context Manager: Getting or creating context ===
[10:00:00] File size: 25,500,000 bytes (25.5 MB)
[10:00:00] Creating new cache...
[10:00:00] Step 1: Uploading file to Gemini File API...
[10:00:01] === UPLOAD FILE START ===
[10:00:01] File path: /var/www/html/modules/ecom_filtersearch/llms.txt
[10:00:01] File size: 25,500,000 bytes (25.5 MB)
[10:00:01] MIME type: text/plain
[10:00:01] Display name: [mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
[10:00:01] Multipart payload size: 25,501,234 bytes
[10:00:01] Upload URL: https://generativelanguage.googleapis.com/v1beta/files?key=***API_KEY***
[10:00:01] Executing CURL upload to Gemini File API...
[10:00:15] Upload completed in 14.23 seconds
[10:00:15] HTTP Response Code: 200
[10:00:15] SUCCESS: File uploaded to Gemini
[10:00:15] File name: files/abc123xyz
[10:00:15] File URI: https://generativelanguage.googleapis.com/v1beta/files/abc123xyz
[10:00:15] File size: 25,500,000 bytes
[10:00:15] === UPLOAD FILE SUCCESS ===
[10:00:15] File uploaded successfully: files/abc123xyz
[10:00:15] File URI: https://generativelanguage.googleapis.com/v1beta/files/abc123xyz
[10:00:15] Storing file info in database...
[10:00:15] Storing file with shop identifier: mitienda-com_mi-tienda-online
[10:00:15] File info stored successfully
[10:00:15] Step 2: Creating cached context...
[10:00:16] === CREATE CACHE START ===
[10:00:16] File URI: https://generativelanguage.googleapis.com/v1beta/files/abc123xyz
[10:00:16] Model: gemini-1.5-pro-002
[10:00:16] TTL: 3600 seconds
[10:00:16] Display name: [mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
[10:00:16] Executing CURL request to create cache...
[10:00:20] Cache creation completed in 4.12 seconds
[10:00:20] HTTP Response Code: 200
[10:00:20] SUCCESS: Cache created in Gemini
[10:00:20] Cache name: cachedContents/xyz789abc
[10:00:20] Display name: [mitienda-com_mi-tienda-online] Product Catalog - 2026-01-13 10:00
[10:00:20] Expires at: 2026-01-13T11:00:20Z
[10:00:20] Storing cache info in database...
[10:00:20] Storing cache with shop identifier: mitienda-com_mi-tienda-online
[10:00:20] === CREATE CACHE SUCCESS ===
[10:00:20] Cache created successfully: cachedContents/xyz789abc
[10:00:20] Cleaning up old resources...
[10:00:20] Cleanup completed
```

Fecha de implementación: 2026-01-13

Versión: 1.4.1

Estado:  Completado y probado

Implementación Completada: Gemini Large Context Manager

✓ Resumen de la Solución

Has solicitado que el modelo de IA pueda recibir archivos de **20MB o más grandes**, y que estos se mantengan en una sesión para todas las consultas futuras, con actualización diaria.

¡Misión cumplida! 🚀

Archivos Creados

1. Classes (3 archivos nuevos)

`classes/GeminiFileManager.php`

- **Función:** Gestiona uploads de archivos grandes a Gemini File API
- **Capacidad:** Hasta 2GB (vs 3MB anterior)
- **Métodos principales:**
 - `uploadFile()` - Sube archivo a Gemini
 - `getFile()` - Obtiene info del archivo
 - `deleteFile()` - Elimina archivo
 - `listFiles()` - Lista todos los archivos

`classes/GeminiCacheManager.php`

- **Función:** Gestiona caches de contexto
- **Duración:** 1 hora (renovable)
- **Métodos principales:**
 - `createCache()` - Crea cache del contexto
 - `getCache()` - Obtiene info del cache
 - `updateCache()` - Renueva TTL del cache
 - `deleteCache()` - Elimina cache
 - `getActiveCacheFromDb()` - Obtiene cache activo

`classes/GeminiLargeContextManager.php` ★ (PRINCIPAL)

- **Función:** Orquesta todo el proceso automáticamente
- **Método principal:** `getOrCreateContext($filePath)`
- **Características:**
 - Detecta si el archivo ha cambiado
 - Sube archivo si es necesario

- Crea/renueva cache automáticamente
- Limpia recursos antiguos
- Genera estadísticas

2. Controllers (1 archivo nuevo)

`controllers/front/cachemanagement.php`

- **Función:** API para gestión manual desde back office
- **Acciones disponibles:**
 - `get_stats` - Ver estadísticas del cache
 - `update_cache` - Actualizar cache manualmente
 - `clear_cache` - Limpiar cache
 - `test_upload` - Probar upload de archivo

3. Cron (1 archivo nuevo)

`cron/update_cache.php`

- **Función:** Actualización automática diaria
- **Ejecución:** Vía crontab (Linux) o Programador de tareas (Windows)
- **Frecuencia recomendada:** Diariamente a las 2:00 AM
- **Características:**
 - Verifica si la actualización es necesaria
 - Actualiza cache si el archivo cambió
 - Limpia recursos antiguos
 - Genera logs detallados

4. Documentación (2 archivos)

`GEMINI_LARGE_CONTEXT_README.md`

- Guía completa de uso
- Instrucciones de configuración
- Troubleshooting
- Análisis de costos
- Ejemplos de código

`CHANGELOG.md` (actualizado)

- Nueva versión 1.4.0
- Changelog detallado de cambios

Modificaciones en Archivos Existentes

`controllers/front/assist.php`

Cambio 1: Método `getContextContent()` **(líneas 239-291)**

Antes: - Archivos > 3MB → Error y fallback a RAG

Ahora: - Archivos > 3MB → Upload a Gemini + Cache automático - Almacena cache name en cookie - Retorna marker especial `[USING_CACHED_CONTEXT]`

Cambio 2: Método `callGemini()` **(líneas 502-530)**

Antes: - Llamada directa a Gemini API

Ahora: - Detecta si hay cache disponible - Si hay cache → Usa `GeminiLargeContextManager` - Si no hay cache → Método normal - Fallback automático si cache expira

Base de Datos

Tablas Creadas Automáticamente

`ps_ecom_gemini_cache`

```
- id (INT)
- cache_name (VARCHAR 255) - Nombre del cache en Gemini
- display_name (VARCHAR 255) - Nombre descriptivo
- model (VARCHAR 100) - Modelo usado
- expire_time (DATETIME) - Fecha de expiración
- created_at (DATETIME) - Fecha de creación
- updated_at (DATETIME) - Última actualización
```

`ps_ecom_gemini_files`

```
- id (INT)
- file_name (VARCHAR 255) - Nombre del archivo en Gemini
- uri (VARCHAR 500) - URI del archivo
- mime_type (VARCHAR 100) - Tipo MIME
- size_bytes (BIGINT) - Tamaño en bytes
- local_path (VARCHAR 500) - Ruta local del archivo
- created_at (DATETIME) - Fecha de creación
- expires_at (DATETIME) - Fecha de expiración (48h)
```

Cómo Funciona (Flujo Automático)

Primera Vez (Archivo > 3MB)

1. **Usuario abre asistente** en home page
2. **Sistema detecta** `llms.txt` > 3MB
3. **Upload automático** del archivo a Gemini File API
4. **Creación de cache** con el archivo subido
5. **Almacenamiento** del cache name en cookie
6. **Respuesta al usuario** usando el cache

Tiempo: ~15-30 segundos (solo primera vez)

Consultas Siguientes

1. **Usuario hace pregunta**
2. **Sistema detecta** cache en cookie
3. **Uso del cache** para la consulta
4. **Respuesta instantánea**

Tiempo: ~2-5 segundos **Costo:** 75% más barato

Renovación Automática

- **Cada 30 minutos** (si hay actividad): Renueva TTL del cache
- **Cada 24 horas** (cron job): Actualiza archivo y cache
- **Cada 48 horas:** Limpia archivos antiguos

Ahorro de Costos

Ejemplo con archivo de 20MB

Archivo de 20MB ≈ 5 millones de tokens

Escenario	Costo por Consulta*	Costo 1000 Consultas*
Sin Cache	\$0.625	\$625.00
Con Cache	\$0.156	\$156.00
Ahorro	75%	\$469.00

*Precios aproximados con Gemini 1.5 Pro

Configuración del Cron Job

Linux / macOS

Edita crontab:

```
crontab -e
```

Añade esta línea:

```
0 2 * * * php /ruta/completa/modules/ecom_filtersearch/cron/update_cache.php
```

Windows

1. Abre **Programador de tareas**
2. Crear tarea básica
3. **Programa:** `php.exe`
4. **Argumentos:** `"Z:\ruta\completa\modules\ecom_filtersearch\cron\update_cache.php"`
5. **Frecuencia:** Diariamente a las 2:00 AM

Uso Manual (Opcional)

Ver Estadísticas

Desde consola del navegador en back office:

```
fetch('/module/ecom_filtersearch/cachemanagement?action=get_stats')  
  .then(r => r.json())  
  .then(data => console.log(data));
```

Actualizar Cache Manualmente

```
fetch('/module/ecom_filtersearch/cachemanagement?action=update_cache')  
  .then(r => r.json())  
  .then(data => console.log(data));
```

Limpiar Cache

```
fetch('/module/ecom_filtersearch/cachemanagement?action=clear_cache')  
  .then(r => r.json())  
  .then(data => console.log(data));
```

Logs

Ubicaciones

- **Asistente:** logs/assist_debug_YYYY-MM-DD.log
- **Cron:** logs/cache_cron_YYYY-MM-DD.log
- **Gestión:** logs/cache_management_YYYY-MM-DD.log

Ejemplo de Log Exitoso

```
[09:00:00] === Large Context Manager: Getting or creating context ===  
[09:00:00] File size: 25,500,000 bytes (25.5 MB)  
[09:00:01] Creating new cache...  
[09:00:01] Step 1: Uploading file to Gemini File API...  
[09:00:15] File uploaded successfully: files/abc123xyz  
[09:00:15] Step 2: Creating cached context...  
[09:00:20] Cache created successfully: cachedContents/xyz789abc
```

Ventajas de la Solución

Responde a tus Requisitos

1.  **Archivos de 20MB o mayores:** Hasta 2GB
2.  **Sesión persistente:** Cache válido 1 hora, renovable
3.  **Todas las consultas futuras:** Sí, usa el mismo cache
4.  **Actualización diaria:** Cron job automático

Beneficios Adicionales

- **75% de ahorro** en costos de API
- **Más rápido** que cargar el archivo cada vez
- **Automático** - no requiere intervención manual
- **Robusto** - maneja errores y expiraciones
- **Monitoreado** - logs detallados
- **Escalable** - soporta múltiples archivos

Verificación

Comprobar que Funciona

1. **Genera un** `11ms.txt` **> 3MB**
2. **Abre el asistente** en la home page

3. **Revisa el log:** `logs/assist_debug_YYYY-MM-DD.log`
 4. **Busca estas líneas:** `Context: Home Page llms.txt LARGE FILE (...) -> Using Gemini File API + Context Caching Cache created/retrieved: cachedContents/...`
 5. **Haz una consulta** al asistente
 6. **Revisa el log nuevamente:** `Using Large Context Manager with cache: cachedContents/...`
-

Soporte

Si tienes problemas:

1. **Revisa los logs** en `modules/ecom_filtersearch/logs/`
 2. **Ejecuta test manual:** `javascript fetch('/module/ecom_filtersearch/cachemanagement?action=test_upload')`
 3. **Verifica configuración:** - API Key válida - Modelo: `gemini-1.5-pro-002` o `gemini-1.5-flash-002` - Provider: Gemini
-

Documentación Completa

Lee `GEMINI_LARGE_CONTEXT_README.md` para: - Guía detallada de uso - Troubleshooting avanzado - Análisis de costos - Ejemplos de código - Configuración avanzada

¡Listo para Usar!

El sistema está **completamente implementado** y **listo para producción**.

No requiere configuración adicional - se activa automáticamente cuando detecta archivos > 3MB.

Recomendación: Configura el cron job para actualizaciones automáticas diarias.

Fecha de implementación: 2026-01-13

Versión del módulo: 1.4.0

Estado:  Completado y probado

Actualización a Versión 1.4.1 - Guía de Upgrade

Resumen

Versión anterior: 1.3.1

Versión nueva: 1.4.1

Fecha: 2026-01-13

Nuevas Características

Soporte Multi-Tienda

- Identificador único por tienda basado en dominio + nombre
- Diferenciación de archivos y caches en Gemini
- Tracking completo en base de datos

Logging Completo

- Registro detallado de todas las operaciones
- Tiempos de ejecución
- Información de errores completa
- Shop identifier en todos los logs

Cambios en Base de Datos

Tabla `ps_ecom_gemini_files`

Nuevas columnas:

```
shop_identifier VARCHAR(255) NOT NULL DEFAULT ''
shop_domain VARCHAR(255) NOT NULL DEFAULT ''
shop_name VARCHAR(255) NOT NULL DEFAULT ''
```

Nuevo índice:

```
INDEX shop_identifier (shop_identifier)
```

Tabla `ps_ecom_gemini_cache`

Nuevas columnas:

```
shop_identifier VARCHAR(255) NOT NULL DEFAULT ''  
shop_domain VARCHAR(255) NOT NULL DEFAULT ''  
shop_name VARCHAR(255) NOT NULL DEFAULT ''
```

Nuevo índice:

```
INDEX shop_identifier (shop_identifier)
```



Proceso de Actualización

Automático (Recomendado)

1. **Subir archivos** al servidor
2. **Ir al back office** → Módulos → Módulos y Servicios
3. **Buscar** "Ecom Filter Search"
4. **Hacer clic** en "Actualizar"
5. **Esperar** confirmación

El script de upgrade se ejecutará automáticamente y: - ☒ Añadirá las nuevas columnas - ☒ Creará los índices - ☒ Actualizará registros existentes con información de la tienda actual - ☒ Registrará todo en logs de PrestaShop

Manual (Si es necesario)

Si la actualización automática falla, ejecuta este SQL manualmente:


```
-- Tabla ps_ecom_gemini_files
ALTER TABLE `ps_ecom_gemini_files`
ADD COLUMN `shop_identifier` VARCHAR(255) NOT NULL DEFAULT '' AFTER `local_path`,
ADD COLUMN `shop_domain` VARCHAR(255) NOT NULL DEFAULT '' AFTER `shop_identifier`,
ADD COLUMN `shop_name` VARCHAR(255) NOT NULL DEFAULT '' AFTER `shop_domain`,
ADD INDEX `shop_identifier` (`shop_identifier`);

-- Tabla ps_ecom_gemini_cache
ALTER TABLE `ps_ecom_gemini_cache`
ADD COLUMN `shop_identifier` VARCHAR(255) NOT NULL DEFAULT '' AFTER `expire_time`,
ADD COLUMN `shop_domain` VARCHAR(255) NOT NULL DEFAULT '' AFTER `shop_identifier`,
ADD COLUMN `shop_name` VARCHAR(255) NOT NULL DEFAULT '' AFTER `shop_domain`,
ADD INDEX `shop_identifier` (`shop_identifier`);

-- Actualizar registros existentes (ajusta los valores según tu tienda)
UPDATE `ps_ecom_gemini_files`
SET `shop_identifier` = 'mitienda-com_mi-tienda',
    `shop_domain` = 'mitienda.com',
    `shop_name` = 'Mi Tienda'
WHERE `shop_identifier` = '';

UPDATE `ps_ecom_gemini_cache`
SET `shop_identifier` = 'mitienda-com_mi-tienda',
    `shop_domain` = 'mitienda.com',
    `shop_name` = 'Mi Tienda'
WHERE `shop_identifier` = '';
```

Verificación Post-Actualización

1. Verificar Versión

Back Office → Módulos → Ecom Filter Search

Debe mostrar: **Versión 1.4.1**

2. Verificar Base de Datos

```
-- Verificar columnas en ps_ecom_gemini_files
SHOW COLUMNS FROM `ps_ecom_gemini_files`;

-- Verificar columnas en ps_ecom_gemini_cache
SHOW COLUMNS FROM `ps_ecom_gemini_cache`;

-- Verificar datos actualizados
SELECT shop_identifier, shop_domain, shop_name FROM `ps_ecom_gemini_files`;
SELECT shop_identifier, shop_domain, shop_name FROM `ps_ecom_gemini_cache`;
```

3. Verificar Logs

Ubicación: Avanzado → Logs

Buscar: - "EcomFilterSearch: Starting upgrade to v1.4.1" - "EcomFilterSearch: Upgrade to v1.4.1 completed successfully"

4. Probar Funcionalidad

1. **Generar llms.txt** (si tienes archivo > 3MB)
2. **Abrir asistente** en la tienda
3. **Revisar logs** en `modules/ecom_filtersearch/logs/`
4. **Verificar** que aparece el shop identifier

Logs del Upgrade

El script de upgrade registra todo en los logs de PrestaShop:

```
[INFO] EcomFilterSearch: Starting upgrade to v1.4.1
[INFO] EcomFilterSearch: Upgrading table ps_ecom_gemini_files
[INFO] EcomFilterSearch: Added shop_identifier column to ps_ecom_gemini_files
[INFO] EcomFilterSearch: Added shop_domain column to ps_ecom_gemini_files
[INFO] EcomFilterSearch: Added shop_name column to ps_ecom_gemini_files
[INFO] EcomFilterSearch: Updated 5 existing records in ps_ecom_gemini_files
[INFO] EcomFilterSearch: Upgrading table ps_ecom_gemini_cache
[INFO] EcomFilterSearch: Added shop_identifier column to ps_ecom_gemini_cache
[INFO] EcomFilterSearch: Added shop_domain column to ps_ecom_gemini_cache
[INFO] EcomFilterSearch: Added shop_name column to ps_ecom_gemini_cache
[INFO] EcomFilterSearch: Updated 3 existing records in ps_ecom_gemini_cache
[INFO] EcomFilterSearch: Upgrade to v1.4.1 completed successfully
```

Notas Importantes

Compatibilidad

-  **Compatible** con versiones anteriores
-  **No rompe** funcionalidad existente
-  **Actualiza** registros existentes automáticamente
-  **Mantiene** todos los datos previos

Tablas Opcionales

Las tablas `ps_ecom_gemini_files` y `ps_ecom_gemini_cache` solo existen si: - Has usado archivos > 3MB - El sistema ha creado caches

Si las tablas no existen, el upgrade las creará cuando sea necesario.

Múltiples Tiendas

Si tienes **múltiples tiendas** en la misma base de datos: - El upgrade se ejecuta **una sola vez** - Cada tienda tendrá su **propio shop_identifier** - Los registros se actualizan con la información de la **tienda actual**

Troubleshooting

Error: "Column already exists"

Causa: Las columnas ya fueron añadidas manualmente

Solución: Ignorar el error, el upgrade continuará

Error: "Table doesn't exist"

Causa: Las tablas aún no han sido creadas

Solución: Normal, se crearán cuando uses archivos > 3MB

Registros no actualizados

Verificar:

```
SELECT * FROM `ps_ecom_gemini_files` WHERE `shop_identifier` = '';  
SELECT * FROM `ps_ecom_gemini_cache` WHERE `shop_identifier` = '';
```

Solución: Ejecutar UPDATE manual (ver sección Manual)

Impacto en Rendimiento

- ✓ **Mínimo:** Solo añade 3 columnas VARCHAR
- ✓ **Índices optimizados:** Búsquedas rápidas por shop_identifier
- ✓ **Sin cambios** en consultas existentes
- ✓ **Backward compatible:** Funciona con y sin datos

Documentación Adicional

- Guía completa:** `MULTI_SHOP_LOGGING_README.md`
- Changelog:** `CHANGELOG.md`
- Implementación:** `IMPLEMENTACION_COMPLETADA.md`

Checklist Post-Upgrade

- ☐ Versión actualizada a 1.4.1
 - ☐ Columnas añadidas en `ps_ecom_gemini_files`
 - ☐ Columnas añadidas en `ps_ecom_gemini_cache`
 - ☐ Índices creados correctamente
 - ☐ Registros existentes actualizados
 - ☐ Logs de upgrade sin errores
 - ☐ Funcionalidad probada
 - ☐ Shop identifier visible en logs
-

Fecha de creación: 2026-01-13

Versión del documento: 1.0

Estado:  Listo para producción